

Uncertainty-Guided Error Repair in Multi-Step Language Model Agents: When to Target, When to Restart

Abstract

When a multi-step language model agent fails at a complex reasoning task, practitioners face a fundamental recovery question: should the agent redo the single step that went wrong, or restart the entire trajectory from scratch? Targeted repair is cheaper, but it requires knowing *which* step failed—and localizing errors in sequential reasoning chains remains an open problem. We present a systematic study of uncertainty-guided repair strategies for ReAct-style agents across three reasoning benchmarks (HotpotQA, FEVER, and MuSiQue), eight open-weight language models spanning three capacity tiers (7B–72B), and 33 repair strategies. Each failed trajectory is scored at every step using five uncertainty metrics, and six localization rules—including three novel *cascade-aware* rules—predict the broken step. We compare four repair paradigms under matched compute budgets: full restart, random-step repair, oracle-targeted repair, and uncertainty-targeted repair. Our analysis reveals that the optimal repair strategy is task-dependent: on multi-hop QA benchmarks with longer reasoning chains, full restart outperforms even oracle-targeted repair, while on fact verification with shorter trajectories, targeted repair with backtracking is superior. Backtracking upstream from the failure point consistently helps across all tasks, confirming that errors propagate forward through reasoning chains. Uncertainty-guided strategies consistently outperform random-step repair, demonstrating that internal model signals carry meaningful information for error localization. All experiments use controlled comparisons with paired statistical tests, three random seeds, and a validated 72B judge for ground-truth annotation.

Introduction

Large language model (LLM) agents interleave reasoning with tool use to answer questions that require gathering evidence across several sources. ReAct (Yao et al. 2023), Reflexion (Shinn et al. 2023), and chain-of-thought prompting (Wei et al. 2022) all follow this pattern, and all perform well on knowledge-intensive benchmarks. They also fail often: in our experiments, a 32B agent answers only 54.8% of sampled HotpotQA questions correctly, leaving hundreds of trajectories in which some intermediate reasoning step went wrong.

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

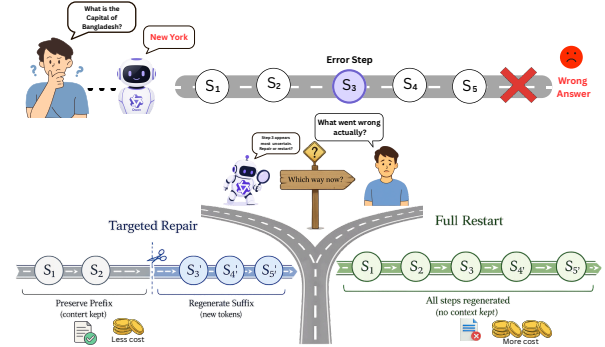


Figure 1: Where should recovery begin? Targeted repair keeps the prefix and regenerates from the predicted broken step; full restart discards the trajectory.

When a trajectory fails, how should it be repaired? Two extremes define the design space (Figure 1):

- **Full restart:** Discard the entire trajectory and regenerate from scratch. Simple, but wasteful, since correct early steps are thrown away.
- **Targeted repair:** Identify the broken step and regenerate only from that point onward. Cheaper, but it requires accurate error localization.

Targeted repair reuses a prefix the agent has already paid for, and prior work has largely assumed it is preferable whenever the broken step can be identified (Madaan et al. 2023; Pan et al. 2024). We test that assumption. On 1,136 failed trajectories from three benchmarks, we repair each trajectory under 126 configurations and three random seeds, holding the regeneration-token budget fixed across strategies. We locate the broken step using step-level uncertainty derived from the model’s own signals, which needs no step-level supervision and no separately trained verifier. Neither extreme is uniformly better. Which strategy wins depends on how long the trajectory is, and on what we call *context contamination*, where erroneous reasoning left in the retained prefix appears to bias the regenerated suffix even when repair begins at the correct step.

Our contributions are:

1. **Three empirical findings** that cut against common as-

sumptions about repair:

- Full restart beats judge-targeted repair on multi-hop QA (14.2% vs. 10.9% on HotpotQA), suggesting that context contamination limits how much surgical repair can help.
 - Backtracking a few steps upstream of the identified failure point improves repair on all three benchmarks, as forward error propagation would predict.
 - Uncertainty-guided repair beats repair from a randomly chosen step even though exact step localization remains weak, which suggests that approximate localization is enough to be useful.
2. **Three cascade-aware localization rules:** cascade-upstream, cascade-gradient, and cascade-weighted. These follow from a simple hypothesis. If errors propagate forward, uncertainty should peak *downstream* of the step where reasoning first went wrong, which makes the peak itself the wrong place to intervene. Section quantifies how often the true error precedes the uncertainty peak.
 3. **A controlled comparison** of 33 strategies (3 baselines, plus 5 uncertainty metrics $\times$ 6 localization rules), extended across backtrack offsets and two nudge types to 126 configurations under matched token budgets, with paired significance tests over three seeds.

Related Work

Multi-Step Reasoning and Tool-Using Agents. Chain-of-thought prompting makes intermediate reasoning explicit and improves performance on multi-step tasks (Wei et al. 2022). ReAct extends this idea to tool-using agents by interleaving reasoning, actions, and observations within a single trajectory (Yao et al. 2023). In this setting, later decisions depend not only on earlier reasoning but also on evidence returned by previous tool calls. Reflection-based agents further reuse feedback from unsuccessful trials as verbal memory to guide later attempts (Shinn et al. 2023). These studies establish the trajectory-based setting considered here. Their primary aim, however, is to generate stronger trajectories or improve subsequent trials rather than determine how much of an already failed trajectory should be preserved during recovery.

Self-Correction and Recovery. Self-correction methods differ in both the source of feedback and the scope of revision. Self-Refine uses the same language model to generate feedback and iteratively revise its output (Madaan et al. 2023), whereas CRITIC grounds verification in external tools before producing a corrected answer (Gou et al. 2024). Intrinsic self-correction remains unreliable on reasoning tasks when informative external feedback is unavailable and may even degrade initially correct responses (Huang et al. 2024). Broader evaluations further show that the effectiveness and computational cost of correction vary across intrinsic, externally guided, and fine-tuned strategies (Tie et al. 2025). Importantly, mistake localization and mistake correction are distinct capabilities: models often correct a reasoning trace when the error location is supplied, despite

struggling to identify that location independently (Tyen et al. 2024). This distinction is central to trajectory repair, since successful recovery may depend less on whether the model can regenerate an answer than on whether intervention begins at an appropriate point.

Recovery has also been studied in tool-interaction settings. Structured reflection policies diagnose erroneous tool calls and propose corrected follow-up actions (Su et al. 2026). Other work evaluates whether agents can construct backup plans after external functions become unavailable (Wang et al. 2025) or adapt workflows under probabilistic tool failures and flawed instructions (Ao et al. 2026). These studies establish recovery and replanning as capabilities distinct from clean-path task execution. Their failures are largely introduced by the environment or tool interface, whereas our setting begins with a naturally failed reasoning-retrieval trajectory and asks where regeneration should begin within that trajectory.

Process Verification and Failure Localization. Process-level evaluation examines intermediate steps rather than judging only the final answer. Process reward models use step-level supervision to distinguish valid from invalid reasoning (Lightman et al. 2024), while verifier benchmarks assess the relevance, evidential support, and logical validity of individual steps (Jacovi et al. 2024). ProcessBench focuses specifically on identifying the first erroneous step in mathematical reasoning (Zheng et al. 2025). Related work further shows that locating the first mathematical error remains difficult even when models are given access to a reference solution (Srivatsa, Maurya, and Kochmar 2025). Together with prior findings on mistake localization (Tyen et al. 2024), this literature suggests that detecting an incorrect final answer and locating the earliest consequential error are substantially different tasks.

Recent preprints extend failure attribution from reasoning chains to full agent trajectories. AgentDebug traces failures to root causes and provides targeted feedback for subsequent attempts (Zhu et al. 2025). AgentRx uses step-wise constraint violations and an LLM judge to identify the first unrecoverable critical failure (Barke et al. 2026), while DRIFT audits claim support and downstream dependencies to localize harmful spans in deep-research trajectories (Wang et al. 2026). The targets used across this literature are not interchangeable: the first incorrect step, the first unrecoverable step, the root cause, and the first harmful span may occur at different positions. Moreover, localization accuracy alone does not establish that the predicted location is the most effective point from which to regenerate the remaining trajectory.

Uncertainty in Language Models and Agent Trajectories. Uncertainty estimation provides a complementary, training-free signal for assessing model outputs. Self-consistency measures agreement across sampled reasoning paths (Wang et al. 2023), while SelfCheckGPT uses disagreement among sampled generations to detect unsupported content (Manakul, Liusie, and Gales 2023). Semantic uncertainty groups meaning-equivalent generations before estimating their variation (Kuhn, Gal, and Farquhar 2023),

and verbalized-confidence methods directly elicit probability judgments from the model (Tian et al. 2023). Most of these approaches operate at the answer or passage level, where the objective is to estimate correctness, hallucination risk, or whether an output should be trusted.

SAUP extends uncertainty estimation to multi-step agents by aggregating uncertainty across reasoning and action steps using situation-dependent weights (Zhao et al. 2025). This demonstrates that final-step uncertainty alone may overlook uncertainty accumulated earlier in a trajectory. However, SAUP produces an overall reliability estimate rather than selecting a point of intervention. Our setting instead retains the step-wise uncertainty profile and examines whether its temporal structure identifies an actionable origin for regeneration. Prior work has examined trajectory generation, self-correction, process verification, uncertainty estimation, error attribution, and recovery under unreliable tools. These directions leave open a narrower but practically important question: given a failed multi-step trajectory, should the system preserve the earlier prefix and regenerate only a suffix, or discard the trajectory and restart from a clean context? We study whether uncertainty signals available from the acting model can select an effective repair origin, whether moving upstream from that location improves recovery when errors propagate, and when targeted repair is preferable to full restart. We compare uncertainty-targeted repair with random-step repair, judge-targeted reference repair, and full restart under matched regeneration-token budgets.

Method

Given a failed ReAct trajectory $\tau = (s_1, \dots, s_T)$, our framework determines where recovery should begin and whether the trajectory should be repaired locally or restarted from the beginning. Each step s_i contains the reasoning, action, action input, and resulting observation produced at that stage. Token log-probabilities are stored during the original trajectory generation. After benchmark-based correctness checking, correct trajectories are excluded and failed trajectories are retained for analysis.

The framework contains three main stages: (1) estimating uncertainty at each trajectory step, (2) converting the resulting uncertainty profile into a predicted failure location, and (3) executing recovery through targeted suffix regeneration, optionally with backtracking and a corrective nudge. Figure 2 summarizes the complete diagnosis, recovery, reference-annotation, and evaluation pipeline.

Step-Level Uncertainty Scoring

For every step s_i , we compute five families of uncertainty signals. Let $\mathcal{T}_i$ denote the set of generated tokens in step s_i , and let ℓ_t denote the log-probability of the sampled token t . Token-based signals are aggregated over the tokens in each step to obtain a scalar step-level uncertainty score.

Top- K Token Entropy. For each generated token t , the inference engine stores the log-probabilities $\{\ell_{t,1}, \dots, \ell_{t,K}\}$ of the top- K candidate tokens. Because only the top- K alternatives are available, we renormalize their probabilities

as

$$\hat{p}_{t,j} = \frac{\exp(\ell_{t,j})}{\sum_{r=1}^K \exp(\ell_{t,r})}. \quad (1)$$

The corresponding truncated token entropy is

$$H(t) = - \sum_{j=1}^K \hat{p}_{t,j} \log \hat{p}_{t,j}. \quad (2)$$

We obtain two step-level variants:

$$U_{\text{ent}}^{\text{mean}}(s_i) = \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} H(t), \quad U_{\text{ent}}^{\text{max}}(s_i) = \max_{t \in \mathcal{T}_i} H(t). \quad (3)$$

The mean captures uncertainty distributed across the step, whereas the maximum captures a highly uncertain individual token.

Sampled-Token Probability Complement. For each generated token, we define uncertainty as the complement of the sampled token probability:

$$u_{\text{stp}}(t) = 1 - \exp(\ell_t). \quad (4)$$

As with entropy, we compute mean and maximum aggregations:

$$U_{\text{stp}}^{\text{mean}}(s_i) = \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} u_{\text{stp}}(t), \quad U_{\text{stp}}^{\text{max}}(s_i) = \max_{t \in \mathcal{T}_i} u_{\text{stp}}(t). \quad (5)$$

Perplexity. We compute per-step perplexity from the mean sampled-token surprisal:

$$U_{\text{ppl}}(s_i) = \min \left\{ \exp \left(-\frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} \ell_t \right), \exp(20) \right\}. \quad (6)$$

The upper bound $\exp(20)$ prevents numerical overflow and limits the influence of extreme values.

Self-Consistency. To measure behavioral disagreement, we resample step s_i independently $n = 5$ times at temperature $\tau = 0.7$, while conditioning on the same preceding trajectory context. Each sample is represented by its (action, action input) pair. Let $a_i^{(r)}$ denote the normalized pair produced by sample r . We define

$$U_{\text{sc}}(s_i) = 1 - \frac{\max_a \sum_{r=1}^n \mathbb{I}[a_i^{(r)} = a]}{n}. \quad (7)$$

The score is zero when all samples select the same action and approaches one as their actions become increasingly inconsistent.

Verbalized Confidence. We additionally ask the model to rate its confidence in step s_i on a scale from 0 to 100 using a dedicated prompt at temperature zero. If c_i is the extracted confidence value, the corresponding uncertainty is

$$U_{\text{vc}}(s_i) = 1 - \frac{c_i}{100}. \quad (8)$$

Together, these signals produce a step-wise uncertainty profile

$$U_{1:T} = (U_1, \dots, U_T) \quad (9)$$

for each metric or aggregation variant.

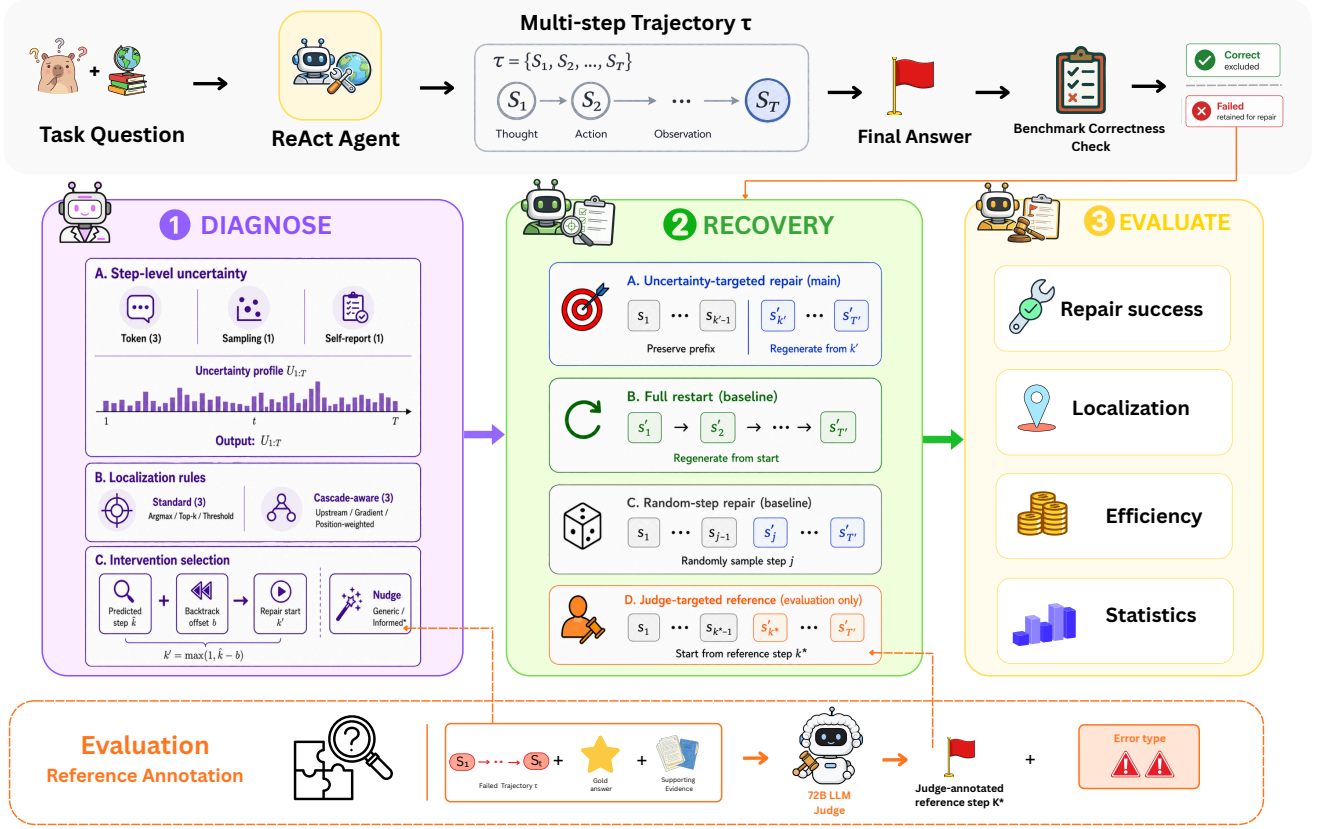


Figure 2: Overview of the uncertainty-guided trajectory repair framework. Failed ReAct trajectories are localized using step-level uncertainty, repaired through targeted suffix regeneration, and compared with full restart, random-step repair, and judge-targeted reference repair. Evaluation measures repair success, localization, efficiency, and statistical significance, with k^* annotated by a 72B LLM judge.

Failure Localization

Given an uncertainty profile $U_{1:T}$, a localization rule predicts a candidate failure step $\hat{k}$. We consider three standard rules and three cascade-aware rules.

Standard Localization Rules

Argmax. The most uncertain step is selected:

$$\hat{k} = \arg \max_{1 \leq i \leq T} U_i. \quad (10)$$

Top- r Earliest. Let $\mathcal{I}_r$ contain the indices of the r largest uncertainty scores. We select the earliest of these candidates:

$$\hat{k} = \min_{i \in \mathcal{I}_r} i. \quad (11)$$

This rule reduces sensitivity to a single downstream uncertainty peak.

Earliest Above Threshold. We calculate the 75th percentile of the uncertainty scores within the trajectory and return the earliest step that exceeds it:

$$\hat{k} = \min \{i : U_i \geq Q_{75}(U_{1:T})\}. \quad (12)$$

Cascade-Aware Localization Rules An upstream error may alter subsequent retrieval, reasoning, or tool use, causing the strongest uncertainty signal to appear after the original failure. In our pilot analysis, approximately 70% of argmax localization misses placed the judge-annotated reference step before the uncertainty peak. We therefore introduce three rules that explicitly favour earlier intervention points.

Cascade-Upstream. Starting from the uncertainty peak, we move back by a fixed look-back distance $L = 2$:

$$\hat{k} = \max \left\{ 1, \arg \max_i U_i - L \right\}. \quad (13)$$

Cascade-Gradient. Instead of selecting the largest uncertainty value, we select the step associated with the largest increase:

$$\hat{k} = \arg \max_{2 \leq i \leq T} (U_i - U_{i-1}). \quad (14)$$

This rule treats a sharp increase as a possible indication that the trajectory has begun to diverge.

Cascade-Weighted. We combine uncertainty magnitude with a positional preference for earlier steps. Let $\tilde{U}_i$ be the min-max normalized uncertainty score and let

$$\widetilde{\text{pos}}_i = \frac{i-1}{T-1} \quad (15)$$

denote normalized trajectory position. We define

$$\text{score}_i = (1-w)\tilde{U}_i + w(1-\widetilde{\text{pos}}_i), \quad (16)$$

with $w = 0.5$, and predict

$$\hat{k} = \arg \max_i \text{score}_i. \quad (17)$$

Repair Execution

The predicted failure location is converted into an operational repair origin. For a selected target step k , we optionally move upstream using a backtracking offset

$$b \in \{0, 1, 2, 3\}. \quad (18)$$

The actual repair origin is

$$k' = \max(1, k - b). \quad (19)$$

Steps $(s_1, \dots, s_{k'-1})$ are retained, while the suffix beginning at $s_{k'}$ is regenerated. Backtracking therefore separates the predicted failure location from the point at which intervention begins.

Recovery Strategies As illustrated in Figure 2, we compare four recovery strategies.

Uncertainty-Targeted Repair. A localization rule selects $k = \hat{k}$ from the step-wise uncertainty profile. After optional backtracking, the prefix is preserved and the trajectory is regenerated from k' onward.

Full Restart. The failed trajectory is discarded and generation restarts from the initial task state. This corresponds to setting the recovery origin to the first trajectory step and retaining no generated prefix.

Random-Step Repair. A trajectory step is selected at random and used as the candidate repair location. This baseline tests whether uncertainty-guided localization provides an advantage over arbitrary intervention.

Judge-Targeted Reference Repair. The repair location is set to the judge-annotated reference step k^* . This strategy provides a reference for evaluating the cost of imperfect localization. Because k^* is produced by an LLM judge rather than a guaranteed ground-truth annotation, we refer to it as a *judge-targeted reference* rather than an oracle.

Corrective Nudges Before generating the first repaired step, the prompt may include one of two corrective nudges.

Generic Nudge. A task-independent message asks the model to reconsider the previous attempt:

“Your previous attempt at this step may have been wrong. Reconsider carefully.”

Informed Nudge. When an error-type label is available, the message provides error-specific guidance. For example, a trajectory labelled with `wrong_search_query` receives:

“Your search query was likely wrong. Try different search terms.”

Reference Annotation and Evaluation

The reference-annotation process is shown in the lower part of Figure 2. For each failed trajectory, a 72B LLM judge receives the trajectory, benchmark gold answer, and supporting evidence. It returns a judge-annotated reference failure step k^* together with an error-type label.

The reference step is used to evaluate localization quality and to construct the judge-targeted repair condition. It is not used by the uncertainty-based localization rules. Recovery strategies are evaluated using repair success, localization quality, computational efficiency, and statistical significance.

Budget Control and Deduplication

Matched Generation Budget. To make targeted repair and full restart comparable, all strategies receive the same regeneration-token budget:

$$B_{\text{repair}} = mB_\tau, \quad m \in \{0.5, 1.0, 2.0\}, \quad (20)$$

where B_τ is the generation-token count of the original failed trajectory. This prevents a strategy from benefiting solely from a larger generation allowance.

Deduplication. Different uncertainty metrics and localization rules may induce the same repair configuration. When multiple strategies select the same trajectory, repair origin, seed, nudge type, and token budget, we execute the repair once and reuse the resulting trajectory for all equivalent configurations. This reduces redundant GPU computation by approximately 70–90% without changing the evaluated outputs.

Experimental Setup

Datasets

We evaluate on three benchmarks spanning distinct reasoning types:

- **HotpotQA** (Yang et al. 2018): 2-hop multi-hop QA over Wikipedia. Distractor setting with 10 paragraphs per question. 500 stratified questions (by level and type). 7,405 total dev questions.
- **FEVER** (Thorne et al. 2018): Fact verification—classify claims as SUPPORTS, REFUTES, or NOT ENOUGH INFO. 500 stratified questions (by label). 19,998 total dev claims.
- **MuSiQue** (Trivedi et al. 2022): 2–4 hop compositional QA designed to defeat shortcut reasoning. 500 stratified questions (by hop count). 2,417 total dev questions.

All datasets use offline document collections (no live retrieval), ensuring reproducibility. Each question’s paragraphs include both supporting and distractor documents.

Models

We use eight open-weight models across three capacity tiers, all with AWQ 4-bit quantization for A100-80GB inference:

- **Small (7–9B)**: Qwen2.5-7B, Llama-3.1-8B, Gemma-2-9B
- **Medium (12–27B)**: Qwen2.5-14B, Mistral-Nemo-12B, Gemma-2-27B
- **Large (70–72B)**: Llama-3.3-70B, Qwen2.5-72B
- **Judge**: Qwen2.5-72B-Instruct-AWQ (for error annotation)

Repair Strategies

We evaluate **33 base strategies**: 3 baselines (full restart, random step, oracle targeted) + 5 metrics $\times$ 6 localization rules. With backtrack offsets $\{0, 2\}$ and nudge types $\{\text{generic}, \text{informed}\}$, this yields **126 total configurations**. Each is run with 3 seeds.

Error Annotation

For each failed trajectory, we obtain ground-truth error labels via a hybrid approach:

1. **Programmatic signal**: Check whether all gold supporting documents were retrieved. The last search step before a retrieval gap is a candidate broken step.
2. **72B judge**: Qwen2.5-72B-Instruct examines the full trajectory and identifies the earliest broken step along with one of five error types: `wrong_search_query`, `wrong_fact_extraction`, `faulty_reasoning`, `premature_or_wrong_answer`, or `formatting_tool_error`.

We validate the judge against 50 human-labeled trajectories per dataset, measuring step-level agreement and Cohen’s κ for error types.

Evaluation Metrics

Repair Success. Fix rate = fraction of failed trajectories where the repair produces a correct answer ($\text{EM} = 1$ or $\text{F1} \geq 0.5$).

Localization Accuracy. Argmax top-1 (exact match), within-1 (off-by-one tolerance), and mean reciprocal rank (MRR) of the oracle step in the rule’s ranking.

Statistical Tests. McNemar’s paired test between strategies (per-question success vectors aggregated via majority vote over 3 seeds), with Holm-Bonferroni correction for multiple comparisons. Bootstrap confidence intervals (10,000 resamples) for all fix rates.

Results

We evaluate recovery using the fix rate over the originally failed trajectories. A repair is considered successful when the regenerated answer satisfies $\text{EM} = 1$ or $\text{F1} \geq 0.5$. Unless otherwise stated, results are aggregated over three random seeds, and confidence intervals are obtained using 10,000 bootstrap resamples.

Table 1: Repair success rates (%) across three benchmarks. Values in parentheses are 95% bootstrap confidence intervals. Bold denotes the best non-ensemble strategy for each dataset.

Strategy	HotpotQA	FEVER	MuSiQue
Random-step repair	8.1 (6.0–10.2)	1.2 (0.6–1.7)	3.8 (2.8–4.9)
Reference-targeted	10.9 (8.6–13.3)	2.5 (1.7–3.3)	4.9 (3.7–6.0)
Reference-targeted + bt2	12.4 (10.0–14.9)	3.2 (2.3–4.0)	5.6 (4.4–6.8)
Reference-targeted + informed	11.4 (9.0–13.9)	2.5 (1.8–3.4)	4.9 (3.8–6.0)
Full restart	14.2 (11.7–16.8)	2.8 (2.0–3.7)	5.9 (4.6–7.2)
Ensemble (any)	36.0 (32.5–39.7)	9.6 (8.1–11.1)	16.2 (14.3–18.3)

Table 2: Effect of two-step backtracking on reference-targeted repair. Values in parentheses denote absolute percentage-point gains.

Strategy	HotpotQA	FEVER	MuSiQue
Reference-targeted	10.9	2.5	4.9
Reference-targeted + bt2	12.4 (+1.5)	3.2 (+0.7)	5.6 (+0.7)

Overall Repair Performance

Table 1 compares the main recovery strategies across the three benchmarks. Full restart achieves the highest single-strategy fix rate on HotpotQA and MuSiQue, reaching 14.2% and 5.9%, respectively. Repair from the judge-annotated reference step obtains 10.9% on HotpotQA and 4.9% on MuSiQue. Moving the repair origin two steps upstream improves these rates to 12.4% and 5.6%, but does not surpass full restart.

The ordering changes on FEVER. Reference-targeted repair with two-step backtracking achieves the highest non-ensemble fix rate of 3.2%, compared with 2.8% for full restart and 2.5% for repair directly from the reference step. Thus, targeted repair and full restart exhibit different strengths across tasks: restart performs best on the two multi-hop QA benchmarks, whereas targeted repair with backtracking performs best on FEVER.

The ensemble condition succeeds when any evaluated member strategy produces a correct answer. It substantially increases the fix rate to 36.0% on HotpotQA, 9.6% on FEVER, and 16.2% on MuSiQue, although at a considerably higher computational cost.

Effect of Backtracking

Table 2 isolates the effect of moving the repair origin two steps upstream from the judge-annotated reference step. Backtracking improves the fix rate on all three benchmarks. The gain is largest on HotpotQA, where performance increases from 10.9% to 12.4%, corresponding to an absolute improvement of 1.5 percentage points. FEVER and MuSiQue both improve by 0.7 points.

Although the gains are modest, their direction is consistent across datasets. Repairing directly from the annotated failure step is therefore not always the most effective intervention. Regenerating a small amount of preceding context can provide a more useful starting state for recovery.

Table 3: Random-step repair compared with the best uncertainty-guided configuration for each dataset.

Strategy	HotpotQA	FEVER	MuSiQue
Random-step repair	8.1	1.2	3.8
Best uncertainty-guided	13.2	3.4	5.6
Absolute gain	+5.1	+2.2	+1.8

Table 4: Exact top-1 localization accuracy relative to the judge-annotated reference step. Dashes denote unavailable results.

Metric	HotpotQA	FEVER	MuSiQue
Self-consistency	0.248	0.223	0.129
Token entropy (max)	0.195	0.210	0.099
Sampled-token uncertainty (max)	0.177	0.212	0.099
Perplexity	0.173	—	—
Verbalized confidence	0.088	—	—
Random-step baseline	0.223	0.304	0.153

Uncertainty-Guided Repair

Table 3 compares random-step repair with the best uncertainty-guided configuration for each benchmark. Uncertainty-guided repair improves the fix rate from 8.1% to 13.2% on HotpotQA, from 1.2% to 3.4% on FEVER, and from 3.8% to 5.6% on MuSiQue. These correspond to absolute gains of 5.1, 2.2, and 1.8 percentage points, respectively.

The gains are observed across all three task types, indicating that the step-wise uncertainty profiles contain information that is useful for choosing where regeneration should begin. This benefit persists despite the relatively low exact-match localization accuracy reported below.

Localization Accuracy

Table 4 reports exact top-1 agreement between the predicted step and the judge-annotated reference step k^* . Self-consistency is the strongest uncertainty-based localizer across all three benchmarks, reaching 0.248 on HotpotQA, 0.223 on FEVER, and 0.129 on MuSiQue. Token-entropy and sampled-token uncertainty perform similarly on FEVER and MuSiQue, while verbalized confidence is substantially weaker on HotpotQA.

Exact localization remains challenging. On FEVER and MuSiQue, the random-step baseline obtains higher exact-match accuracy than the individual uncertainty metrics. Nevertheless, uncertainty-guided repair still outperforms random-step repair in terms of final fix rate. This difference indicates that exact agreement with k^* and operational repair utility are not equivalent. A predicted step can support successful regeneration even when it does not exactly match the reference annotation.

Cost–Accuracy Trade-off

Table 5 reports the average number of generated tokens required by representative repair strategies. A single targeted or restart-based repair requires between 157 and 348 tokens

Table 5: Cost–accuracy trade-off. Token counts are averaged over repair attempts. The ensemble succeeds when any member strategy produces a correct answer.

Strategy	Avg. tokens	Fix rate (%)
<i>HotpotQA</i>		
Random-step repair	182	8.1
Reference-targeted	236	10.9
Full restart	274	14.2
Ensemble (any)	12,868	36.0
<i>FEVER</i>		
Random-step repair	157	1.2
Full restart	221	2.8
Ensemble (any)	10,600	9.6
<i>MuSiQue</i>		
Random-step repair	216	3.8
Full restart	348	5.9
Ensemble (any)	15,317	16.2

on average, depending on the benchmark and strategy. Full restart is more expensive than random-step repair because it regenerates the complete trajectory, but its cost remains close to that of a single targeted intervention.

The ensemble produces substantially higher fix rates but requires 12,868 tokens per HotpotQA example, 10,600 for FEVER, and 15,317 for MuSiQue. This is approximately 47–70 times the cost of the corresponding individual strategies. The ensemble therefore represents a high-compute upper envelope rather than an efficient default repair policy.

Generic and Error-Specific Nudges

Error-specific nudges do not consistently improve repair over a generic reconsideration prompt. On HotpotQA, reference-targeted repair with an informed nudge achieves 11.4%, compared with 10.9% for direct reference-targeted repair and 12.4% when two-step backtracking is applied. On FEVER, the informed condition remains at 2.5%, below the 3.2% obtained with backtracking. On MuSiQue, the informed condition reaches 4.9%, matching direct reference-targeted repair but remaining below both the backtracked variant and full restart.

These results suggest that modifying the intervention point has a larger and more consistent effect than supplying an error-type-specific instruction in the current setting.

Analysis

Repair versus Restart. Full restart performs best on HotpotQA and MuSiQue, whereas reference-targeted repair with backtracking performs best on FEVER. This pattern is consistent with harmful information being retained in longer failed prefixes: earlier reasoning, retrieval decisions, or observations may continue to influence the regenerated suffix. Restart removes this history and allows a different reasoning path. However, because trajectory length, task type, and repair conditions vary together, the current results do not establish retained context as the sole cause.

Localization versus Intervention. Two-step backtracking improves reference-targeted repair across all three benchmarks, suggesting that the most useful repair origin may precede the step at which failure becomes visible. This supports the motivation for cascade-aware localization, which accounts for downstream uncertainty produced by earlier errors. Exact localization remains weak, and random selection sometimes matches the reference step more often than individual uncertainty metrics. Nevertheless, uncertainty-guided repair consistently achieves higher fix rates than random-step repair. Thus, exact agreement with k^* and repair utility measure different properties: uncertainty may identify an unstable region that supports successful regeneration without locating the annotated step exactly.

Practical Trade-off. No individual strategy recovers all failures. Ensemble recovery achieves substantially higher fix rates because different strategies solve different subsets of trajectories, but its token cost is $47\text{--}70\times$ higher than that of individual repairs. A practical system should therefore select between targeted repair and restart using trajectory-level signals such as length, uncertainty concentration, evidence coverage, and estimated prefix reliability, rather than executing every strategy.

Discussion

Implications for Agent Repair. Our results show that exact agreement with the annotated failure step does not necessarily lead to the best repair outcome. The most useful intervention point may lie upstream of the visible failure, which explains the consistent benefit of backtracking. This distinction between diagnostic localization and actionable repair should be considered when designing self-repairing agents.

Full restart performs best on HotpotQA and MuSiQue, while targeted repair with backtracking performs best on FEVER. This pattern is consistent with harmful information remaining in longer failed prefixes, although the current experiments do not isolate retained context as the sole cause. Future repair systems could therefore preserve only verified observations, summarize the valid prefix, or regenerate in a separate context window.

Adaptive Recovery. No single strategy is optimal across all tasks. A practical recovery policy should choose between targeted repair and restart using signals such as trajectory length, uncertainty concentration, evidence coverage, and estimated prefix reliability. Ensemble recovery achieves higher fix rates but is substantially more expensive, making selective strategy choice preferable to exhaustive execution.

Limitations. Our evaluation uses offline document collections and only ReAct-style agents, which limits generalization to open-ended and alternative agent architectures. The reference failure step is produced by a validated 72B judge and may still contain systematic annotation errors. Moreover, the repair conditions are not fully symmetric: retained prefix steps reduce the remaining step allowance, and full restart does not receive the same nudge as targeted repair.

These factors may partially explain the observed advantage of restart and should be controlled in future experiments.

Conclusion

We presented a systematic study of recovery strategies for failed multi-step language-model agents. The framework uses step-level uncertainty to select candidate repair locations and compares targeted suffix regeneration with random-step repair, judge-targeted reference repair, and full restart under matched regeneration budgets.

The results show that recovery depends on more than identifying a single erroneous step. Full restart achieves the strongest single-strategy performance on HotpotQA and MuSiQue, while targeted repair with upstream backtracking performs best on FEVER. Backtracking improves reference-targeted repair across all three benchmarks, and uncertainty-guided intervention consistently produces higher fix rates than random-step repair despite weak exact localization.

Together, these results separate *localization accuracy* from *repair utility*: uncertainty need not identify the exact judge-annotated step to provide a useful regeneration region. They also suggest that the reliability of the retained prefix is central to the choice between local repair and clean restart. Future work should develop adaptive recovery policies that jointly consider trajectory structure, uncertainty, evidence coverage, retained-context quality, and repair cost.

References

- Ao, R.; Min, Z.; Zhu, T.; Yin, W.; and Wang, X. 2026. ResiliBench: Evaluating Agentic Workflow Adaptation in Stochastic Environments. In *International Conference on Learning Representations*.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. *arXiv preprint arXiv:2602.02475*. Preprint.
- Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *The Twelfth International Conference on Learning Representations*.
- Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations*.
- Jacovi, A.; Bitton, Y.; Bohnet, B.; Herzig, J.; Honovich, O.; Tseng, M.; Collins, M.; Aharoni, R.; and Geva, M. 2024. A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 4615–4634.
- Kuhn, L.; Gal, Y.; and Farquhar, S. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *The Eleventh International Conference on Learning Representations*.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; and Sutskever, I. 2024.

Let’s Verify Step by Step. In *The Twelfth International Conference on Learning Representations*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoy, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Manakul, P.; Liusie, A.; and Gales, M. J. F. 2023. Self-CheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Pan, L.; Saxon, M.; Xu, W.; Nathani, D.; Wang, X.; and Wang, W. Y. 2024. Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Automated Correction Strategies. In *Transactions of the Association for Computational Linguistics (TACL)*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Srivatsa, K. A.; Maurya, K. K.; and Kochmar, E. 2025. LLMs Cannot Spot Math Errors, Even When Allowed to Peek into the Solution. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 10914–10928.

Su, J.; Wan, Y.; Yang, J.; Shi, H.; Han, T.; Qiu, Y.; and Luo, J. 2026. Failure Makes the Agent Stronger: Enhancing Accuracy through Structured Reflection for Reliable Tool Interactions. In *Findings of the Association for Computational Linguistics: ACL 2026*, 12712–12734.

Thorne, J.; Vlachos, A.; Christodoulopoulos, C.; and Mittal, A. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Tian, K.; Mitchell, E.; Zhou, A.; Sharma, A.; Rafailov, R.; Yao, H.; Finn, C.; and Manning, C. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 5433–5442.

Tie, G.; Yuan, Z.; Zhao, Z.; Hu, C.; Gu, T.; Zhang, R.; Zhang, S.; Wu, J.; Tu, X.; Jin, M.; Wen, Q.; Chen, L.; Zhou, P.; and Sun, L. 2025. Can LLMs Correct Themselves? A Benchmark of Self-Correction in LLMs. In *Advances in Neural Information Processing Systems*, volume 38.

Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single Hop Question Composition. In *Transactions of the Association for Computational Linguistics (TACL)*, volume 10, 539–554.

Tyen, G.; Mansoor, H.; Carbune, V.; Chen, P.; and Mak, T. 2024. LLMs Cannot Find Reasoning Errors, but Can Correct Them Given the Error Location. In *Findings of the As-*

sociation for Computational Linguistics: ACL 2024, 13894–13908.

Wang, A.; Hager, S.; Asija, A.; Khashabi, D.; and Andrews, N. 2025. Hell or High Water: Evaluating Agentic Recovery from External Failures. In *Conference on Language Modeling*.

Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. *arXiv preprint arXiv:2606.02060*. Preprint.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

Zhao, Q.; Li, D.; Liu, Y.; Cheng, W.; Sun, Y.; Oishi, M.; Osaki, T.; Matsuda, K.; Yao, H.; Zhao, C.; Chen, H.; and Zhao, X. 2025. Uncertainty Propagation on LLM Agent. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 6064–6073.

Zheng, C.; Zhang, Z.; Zhang, B.; Lin, R.; Lu, K.; Yu, B.; Liu, D.; Zhou, J.; and Lin, J. 2025. ProcessBench: Identifying Process Errors in Mathematical Reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 1009–1024.

Zhu, K.; Liu, Z.; Li, B.; Tian, M.; Yang, Y.; Zhang, J.; Han, P.; Xie, Q.; Cui, F.; Zhang, W.; Ma, X.; Yu, X.; Ramesh, G.; Wu, J.; Liu, Z.; Lu, P.; Zou, J.; and You, J. 2025. Where LLM Agents Fail and How They Can Learn from Failures. *arXiv preprint arXiv:2509.25370*. Preprint.